



**Universidad Nacional de La Matanza Departamento de
Ingeniería e Investigaciones Tecnológicas**

Programación Concurrente Investigación Librería Asyncio

**Fecha de Presentación:
3/12/2025**

Docentes:

**Esteban Carnuccio
Dario Hirschfeldt
Palomo Máximo Facundo
Mariano Leonardo Volker**

Integrantes

DNI	Nombre	Apellido	Email
43871283	Fabrizio Javier	Martinez Solomita	fmartinez@alumno.unlam.edu.ar

1	Introducción y Descripción General	3
2	Objetivos de la investigación	3
3	Desarrollo	4
4	Conclusión	6
5	Bibliografía	6

Investigación Librería Asyncio

1 Introducción y Descripción General

En el presente trabajo se abordarán los principales conceptos relacionados con la programación asincrónica en Python utilizando la librería asyncio. Se presentarán las características fundamentales de las funciones definidas con `async`, el uso de la instrucción `await`, y el rol del *event loop* dentro de este modelo de ejecución. Además, se analizará la función `asyncio.create_task()` como mecanismo para gestionar tareas concurrentes. A lo largo del documento se explorarán estos elementos con el fin de comprender su aplicación en programas que requieren eficiencia y concurrencia.

2 Objetivos de la investigación

El objetivo general de este trabajo es comprender el funcionamiento de la programación asincrónica en Python mediante el uso de la librería asyncio. Para ello, se plantean los siguientes objetivos:

Identificar los elementos principales que conforman el modelo asincrónico de Python.

Reconocer la función y estructura de las palabras clave `async` y `await`.

Describir el papel del event loop dentro del flujo de ejecución asincrónico.

Analizar el uso de `asyncio.create_task()` como herramienta para gestionar tareas concurrentes.

Establecer las ventajas y desventajas generales de aplicar asincronía en el desarrollo de software.

3 Desarrollo

La librería `asyncio` se basa en un componente fundamental llamado `event loop`, que actúa como el encargado de coordinar la ejecución de todas las tareas asincrónicas de un programa. Este `event loop` funciona como un planificador central: recibe corutinas, decide cuándo deben ejecutarse, cuándo detenerse y cuándo reanudarse, permitiendo que múltiples operaciones avancen sin bloquear el hilo principal. Gracias a este mecanismo, un solo hilo puede mantener activa una gran cantidad de tareas que dependen de operaciones de entrada y salida, temporizadores o eventos externos, evitando tiempos muertos y aprovechando al máximo la concurrencia cooperativa que propone Python. Dentro de la librería `asyncio` la forma de crear, configurar y eliminar el `event loop` es mediante la función `asyncio.run()` la cual maneja la verdadera creación del `loop` internamente y las condiciones de su finalización.

Sobre esta estructura se construyen las corutinas definidas con la palabra clave `async`, que representan funciones diseñadas para suspender su ejecución de manera controlada. Para realizar esta suspensión utilizan la instrucción `await`, que cede temporalmente el control al `event loop` mientras una operación está pendiente. Este modelo permite que mientras una tarea espera, el programa siga avanzando con otras tareas, logrando así un flujo de trabajo no bloqueante y altamente eficiente. Es decir, `async` y `await` se apoyan directamente en el funcionamiento del `event loop`, ya que dependen de él para decidir cuándo continuar su ejecución. Además es importante remarcar que si bien podría decirse que se bloquea el flujo actual sobre el cual se ejecuta el `async`, realmente no se está bloqueando el hilo sobre el cual ejecuta ya que lo que sucede en una función asíncrona cuando se utiliza `await()` es que se cede el control al `event loop` hasta que la función que está en espera finalice o hasta que se vuelva a dar permiso a esa tarea para ejecutar.

Ejemplo:

Async def operación()

 Print("Comienzo")

 Await asyncio.sleep(1)

 Print("Finalizo")

 Return

Lo que sucederá al ejecutarse este código es que en el momento de que la función `operación` ejecute el `await`, el `event loop` de `asyncio` tomara el control del hilo o lo cederá a otra tarea mientras se espera que se resuelva la operación que corresponde, en este caso un `sleep(1)` solo con fines didácticos, pero generalmente la operación a resolver es una de entrada y salida. Si en este caso no hay otra tarea pendiente a la cual el `event loop` pueda darle permiso para ejecutar, entonces en ese caso la ejecución sería secuencial ya que al terminar de ejecutar el `sleep`, seguirá con la próxima línea.

Por otro lado la función `asyncio.create_task()` permite tomar una corutina y registrarla como una tarea independiente dentro del event loop, haciendo posible que se ejecute de forma concurrente con otras tareas. Esto resulta particularmente útil en situaciones donde se deben lanzar múltiples operaciones similares como solicitudes de red, esperas temporizadas o procesamiento escalonado sin necesidad de esperar a que una termine para comenzar la siguiente. De esta manera, `create_task()` contribuye a maximizar la concurrencia lógica y a mejorar la capacidad de respuesta del programa. Es importante hacer énfasis en concurrencia lógica ya que esta no es una concurrencia tal y como la conocemos en la que existen varios procesos/hilos compitiendo por el uso del procesador, en este caso concurrencia lógica se refiere a una competencia entre diferentes tareas, pero dentro de un mismo hilo, en este caso dicha competencia es administrada por el event loop.

Ejemplo:

```
Async def funcion(param)
```

```
    While (Resultado)
```

```
        Resultado = await operacion_ES(param)
```

```
async def main():
```

```
    task1 = asyncio.create_task(funcion(param1))
```

```
    task2 = asyncio.create_task(funcion(param2)))
```

```
    task3 = asyncio.create_task(funcion(param3)))
```

```
    resultados = await asyncio.gather(task1, task2, task3)
```

```
if __name__ == "__main__":
```

```
    asyncio.run(main())
```

Supongamos que `funcion()` realiza operaciones de entrada y salida continuamente usando el parámetro que se le pasa a la función para identificar el destino, lo que sucede es que cuando una de las task encuentre un `await` en su camino, se frenará mientras espera a que la operación de E/S termine, lo que permite al event loop ceder el control de ejecución a la siguiente tarea que está esperando a ser atendida por lo que no se podrá asegurarse el orden de las ejecuciones ya que esto dependerá de cuanto tarde el dispositivo de E/S en responder, por eso es que se dice que se genera una concurrencia lógica ya que si bien existe una competición de las tareas entre sí, estas compiten por el uso del hilo y no por el uso del procesador y es justamente por lo que se tiene que tener especial cuidado con la ejecución del hilo principal, ya que si en el ejemplo mostrado no se esperara mediante la función `await asyncio.gather()` a que las 3 task terminen, podría darse el caso de que el hilo principal que mantiene con vida la ejecución termine, no logrando así que se completen las operaciones de E/S

necesarias ya que lo que mantiene vivo el hilo es la ejecución de la función especificada en `asyncio.run()`, que en este caso es el `main()`.

Es importante remarcar que el uso de la librería `async` usa un tipo de planificación no tan común en estos días el cual es una planificación cooperativa a nivel de aplicación (no de Sistema operativo) entre las diferentes tareas ya que estas son las que voluntariamente mediante el `await` ceden el control del hilo al event loop el cual funciona como una especie de orquestador, a diferencia de la planificación preemptive donde no existe tal cooperación, sino que es el planificador el que se encarga de administrar a las tareas como en las soluciones que utilizan distintos hilos y proceso que necesitan sincronizarse.

Lo explicado anteriormente sirve para entender las principales ventajas y desventajas que el uso de `async` y `await` tiene para ofrecernos. Como principales ventajas podemos mencionar que se tiene un control totalmente eficiente sobre el control de un único hilo, lo que permite sacarle el máximo provecho a las operaciones de E/S, además el código resultante es más fácil de mantener que uno que requiere una sincronización mediante mecanismos de sincronización que son más avanzados. Como ultima ventaja y la más importante es que no existe un cambio de contexto como tal ya que siempre está ejecutando un único hilo, si bien si hay un pequeño overhead al cambiar la tarea que tiene el control de ejecución, este es mínimo.

Como principales desventajas podemos mencionar que: este tipo de programación solamente es eficiente cuando hay una alta presencia de operaciones de E/S ya que una tarea batch no utilizaría `await` y podría tardar demasiado, lo que tomaría el control del hilo y nunca lo cedería, además se tiene que controlar que todas las tareas que vayan a ser ejecutadas cedan el control de ejecución utilizando `await` en algún momento ya que si no nuevamente podría quedarse bloqueada la ejecución solo en esa tarea.

4 Conclusión

La programación asincrónica con `asyncio` es una herramienta que permite aprovechar al máximo un solo hilo gestionando tareas de forma concurrente y cooperativa. Gracias al event loop, las corutinas pueden avanzar mientras otras esperan, evitando bloqueos y reduciendo el costo de cambios de contexto. Este modelo es ideal para aplicaciones con mucha E/S, ya que ofrece eficiencia y un código más claro que los enfoques multihilo.

Sin embargo, su rendimiento disminuye frente a tareas pesadas de CPU, y requiere que todas las funciones cedan el control usando `await` para no bloquear el flujo.

En resumen, `asyncio` es una solución muy eficaz para concurrencia basada en E/S que bien usada puede ser una herramienta muy poderosa, pero es importante entender en qué casos es óptimo utilizarla y cuando no. Como es costumbre en materias de este estilo, la utilización de esta herramienta dependerá de la tarea que se quiera realizar ya que no existe ni existirá una herramienta método o proceso que solucione todos los problemas existentes, por lo que es nuestro trabajo como programadores elegir la herramienta que mejor se adapte a nuestro problema.

5 Bibliografía

Python Software Foundation, "asyncio — E/S Asíncrona — Documentación de Python en Español, versión 3.12.0" 2024. [En línea]. Disponible en: asyncio — E/S Asíncrona — Documentación de Python en Español -- 3.12.0}

